

Module 5: Graphs.

- 1.1. Introduction. Graph - Terminologies, Representation of Graph.
- 1.2. Graph Traversals - Depth First Search (DFS) and Breadth First Search (BFS).
- 1.3. Graph Application - Topological Sorting.

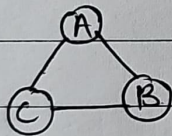
* Introduction

Graph is a non-linear datastructure. It consists of vertices and edges. It is similar to ~~root~~ tree datastructures.

For Graphs

Vertices

Edges

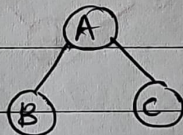


Can form
a cycle

For trees.

nodes.

Vertices



Cannot form
a cycle

* Graph Terminologies

1. **Vertex** - Vertex or Vertices are basically similar to nodes of a tree.
2. **Edges** - Edges are drawn or used to connect two node of the graph.
3. **Order** - Number of vertices contained in the graph.

PAGE No.	
DATE	/ /

* Types of Graph

1. Trivial Graph - Graph having only a single vertex,

(2)

2. Null Graph - Graphs having no edges

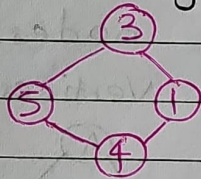
(3)

(5)

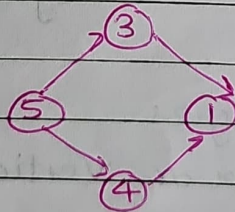
(1)

(4)

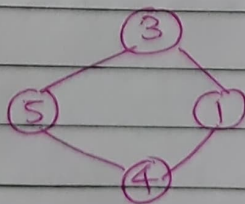
3. Undirected Graph - Edges having no direction



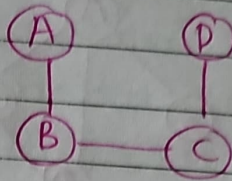
4. Directed Graph - Edges having direction



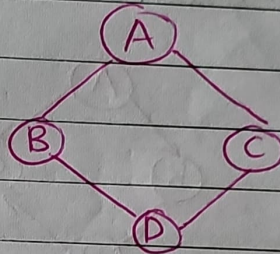
5. Connected Graph - The graph in which from one node, we can visit any other node in the graph



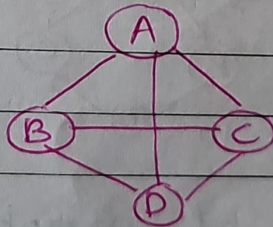
6. **Disconnected Graph** - The graph in which atleast one node is not reachable from a node



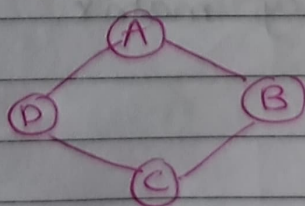
7. **Regular Graph** - The graph in which degree of every vertex is equal to the other vertices of the graph.



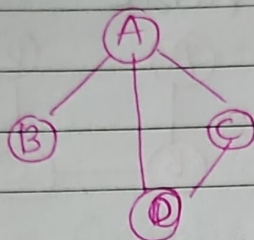
8. **Complete graph** - A graph in which from each node there is an edge to every other node



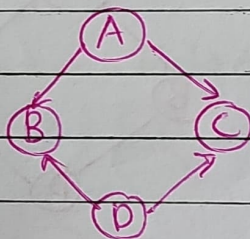
9. **Cycle graph** - Graph is a cycle itself, the degree of each vertex is 2



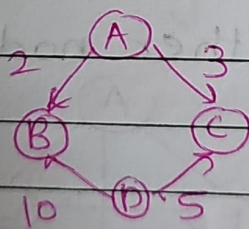
10. Cyclic Graph - A graph containing at least one cycle



11. Directed acyclic graph - Graph which has directions (arrows) but does not contain a cycle



12. Weighted graph - A graph in which edges have weights.



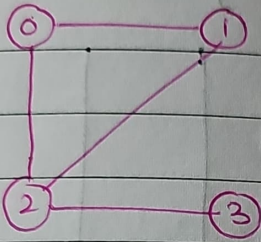
* Representation of graph in memory
 Graphs can be represented using the following methods:

1. Adjacency list
2. Adjacency matrix.

1) Adjacency matrix

In an adjacency matrix, a graph is stored in the form of a 2D matrix where rows and columns denote vertices.

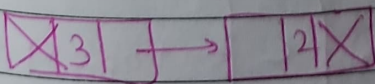
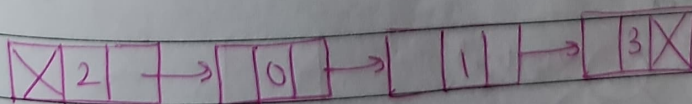
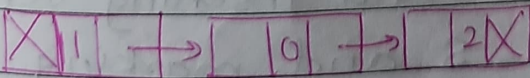
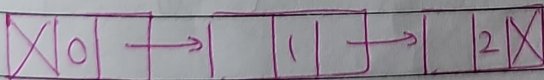
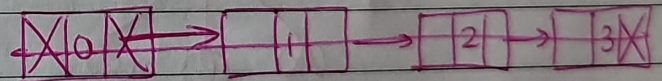
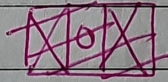
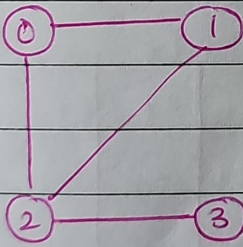
Each entry in the matrix represents the weights of the edge between those vertices.



	0	1	2	3
0	0	1	1	0
1	1	0	1	0
2	1	1	0	1
3	0	0	1	0

2) Adjacency list

Graph is represented as a collection of linked lists. There is an array pointer which points to the edges connected to that vertex.

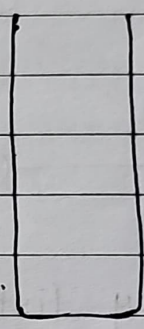
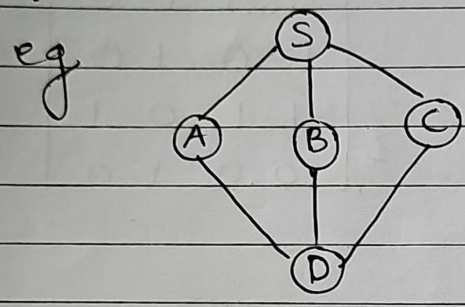


* Graph Traversal Techniques

- 1 BFS - Breadth First search
- 2 Depth First Search - DFS

1 Depth First Search (DFS)

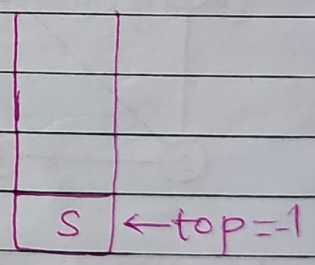
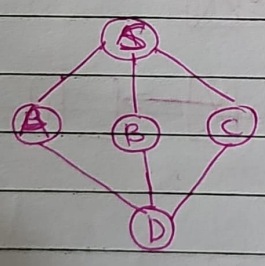
DFS is implemented using stack.



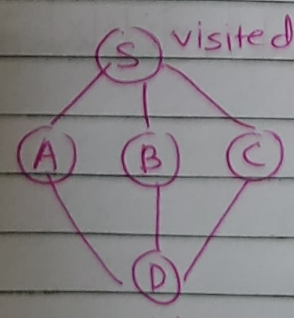
Stack

top = -1 / NULL

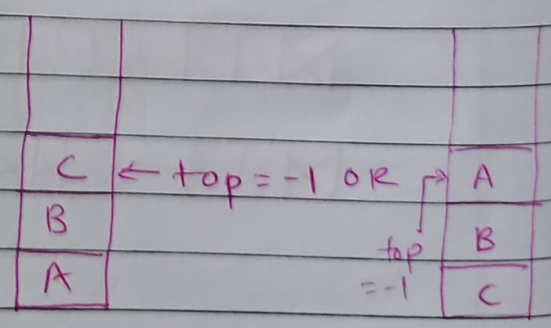
Assuming S to be the start node
Pushing S to the stack.



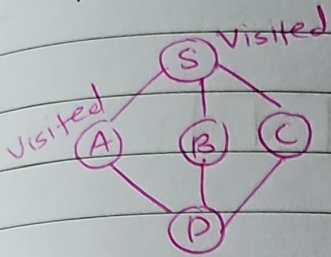
Popping out S and inserting vertices connected to S and marking S as visited



O/P = S



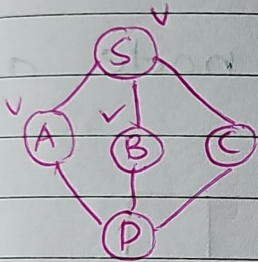
Popping out the top element and inserting the element connected to it, if no element is present pop out the other elements.



top →	D		D ← top.
	B		B
	A		C

O/p = SC

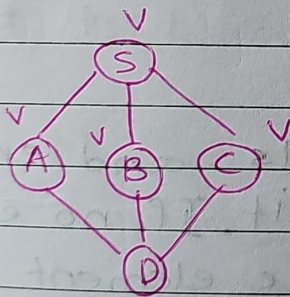
O/p = SA



top →	B		B ← top
	A		C

O/p = SC D

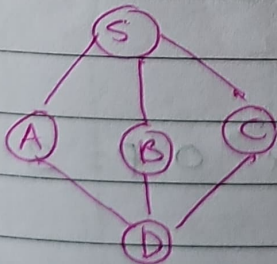
SAD



top →	A		C ← top
-------	---	--	---------

O/p = SCDB

SADB



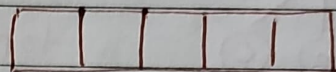
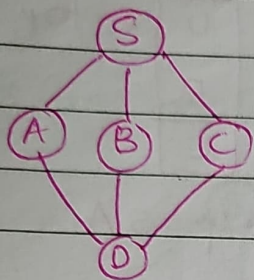
top = -1

O/p = SCDBA

SADBC

2 Breadth First Search

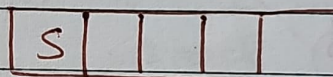
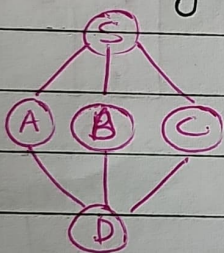
Breadth First Search is implemented using a ~~stack~~ queue



$F = -1$

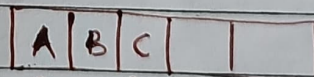
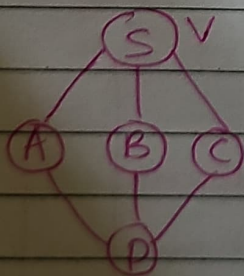
$R = -1$

Assuming S to be the start node and enqueueing it.



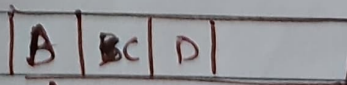
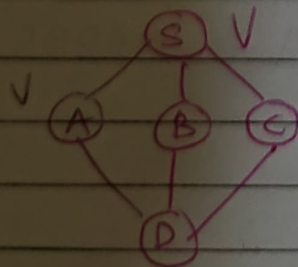
↑↑
FR

Dequeuing the elements and enqueueing elements connected to it. If no element is connected, dequeue the element



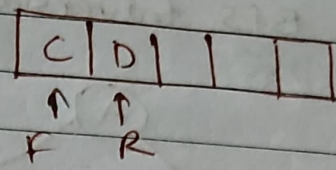
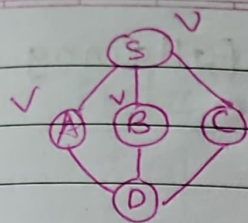
↑ ↑
F R

~~sum~~ O/P = S

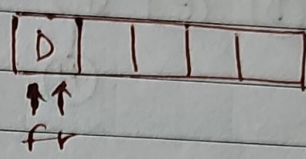
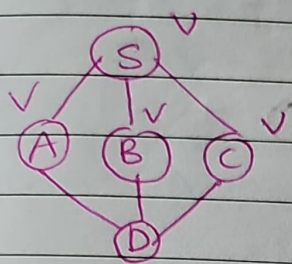


↑ ↑
F R

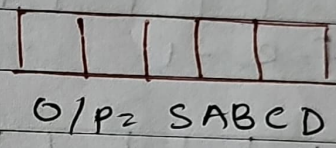
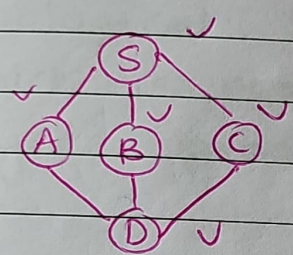
O/P = SA



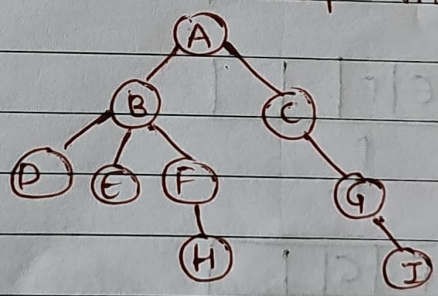
O/P = SAB



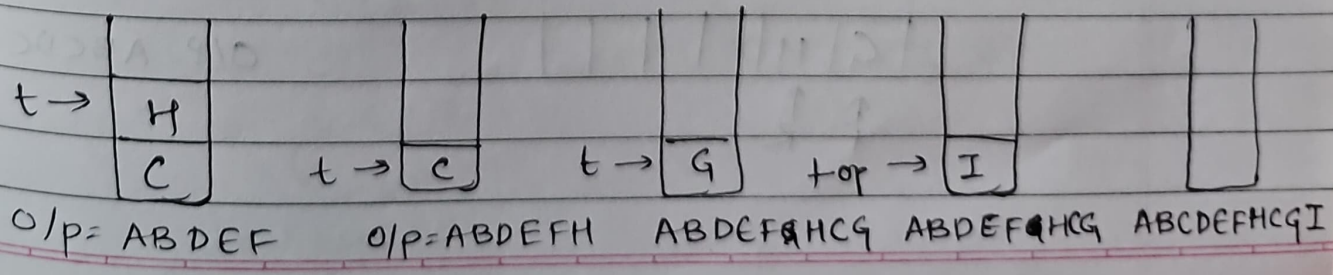
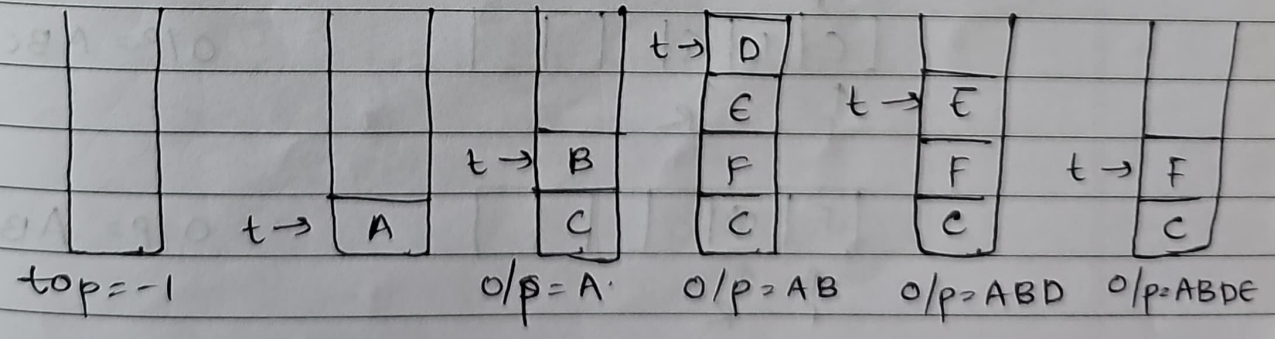
O/P = SABCD



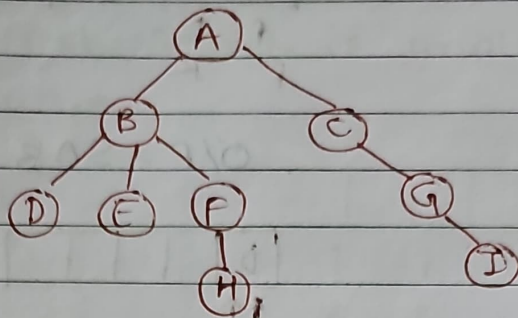
Q Find DFS traversal of the given graph



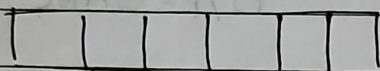
Ans



Q2. Perform BFS traversal on the following graph.



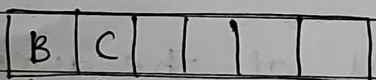
Ans



$f = R = 1$

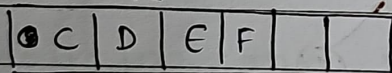


↑ ↑
f R



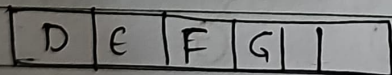
↑ ↑
f R

O/P = A



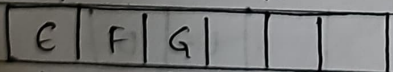
↑ ↑
f R

O/P = AB



↑ ↑
f R

O/P = ABC



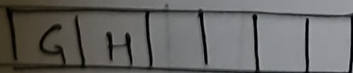
↑ ↑
f R

O/P = ABCD



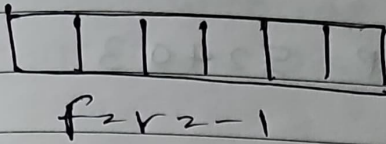
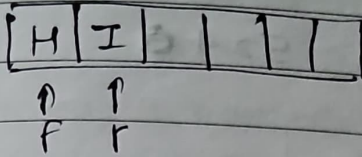
↑ ↑
f R

O/P = ABCDE



↑ ↑
f R

O/P = ABCDEF



O/P = ABCDEFG

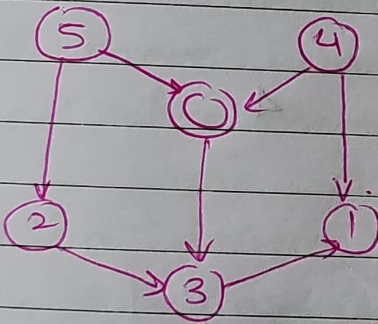
②

O/P = ABCDEFGHI

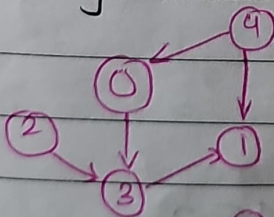
* Graph Applications - Topological Sorting

Topological Sorting is a graph application which is possible if the graph is directed acyclic graph. It starts by vertex having indegree 0.

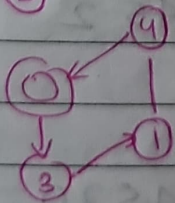
eg



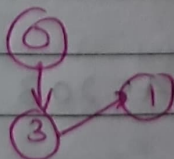
Starting with node 5.



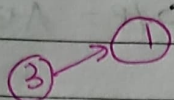
O/P = 5.



O/P = 5 2



O/P = 5 2 4



O/P - 5240

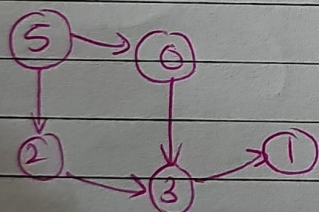
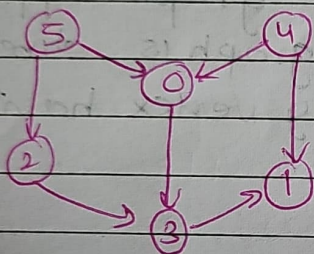


O/P = 52403

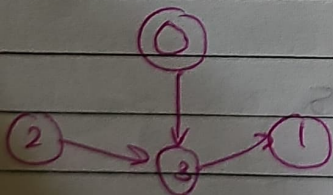
O/P = 524031

Also, 2nd way

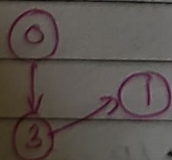
O/P.



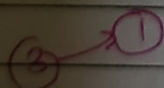
4



45



452



4520



45203

452031